

arXiv:2408.09869v5 [cs.CL] 9 Dec 2024

Docling Technical Report

Version 1.0

Christoph Auer Maksym Lysak Ahmed Nassar Michele Dolfi Nikolaos Livathinos
Panos Vagenas Cesar Berrospi Ramis Matteo Omenetti Fabian Lindlbauer
Kasper Dinkla Lokesh Mishra Yusik Kim Shubham Gupta Rafael Teixeira de Lima
Valery Weber Lucas Morin Ingmar Meijer Viktor Kuropiatnyk Peter W. J. Staar

AI4K Group, IBM Research
Rüschlikon, Switzerland

Abstract

This technical report introduces *Docling*, an easy to use, self-contained, MIT-licensed open-source package for PDF document conversion. It is powered by state-of-the-art specialized AI models for layout analysis (DocLayNet) and table structure recognition (TableFormer), and runs efficiently on commodity hardware in a small resource budget. The code interface allows for easy extensibility and addition of new features and models.

1 Introduction

Converting PDF documents back into a machine-processable format has been a major challenge for decades due to their huge variability in formats, weak standardization and printing-optimized characteristic, which discards most structural features and metadata. With the advent of LLMs and popular application patterns such as retrieval-augmented generation (RAG), leveraging the rich content embedded in PDFs has become ever more relevant. In the past decade, several powerful document understanding solutions have emerged on the market, most of which are commercial software, cloud offerings [3] and most recently, multi-modal vision-language models. As of today, only a handful of open-source tools cover PDF conversion, leaving a significant feature and quality gap to proprietary solutions.

With *Docling*, we open-source a very capable and efficient document conversion tool which builds on the powerful, specialized AI models and datasets for layout analysis and table structure recognition we developed and presented in the recent past [12, 13, 9]. Docling is designed as a simple, self-contained python library with permissive license, running entirely locally on commodity hardware. Its code architecture allows for easy extensibility and addition of new features and models.

Here is what Docling delivers today:

- Converts PDF documents to JSON or Markdown format, stable and lightning fast
- Understands detailed page layout, reading order, locates figures and recovers table structures
- Extracts metadata from the document, such as title, authors, references and language
- Optionally applies OCR, e.g. for scanned PDFs
- Can be configured to be optimal for batch-mode (i.e high throughput, low time-to-solution) or interactive mode (compromise on efficiency, low time-to-solution)
- Can leverage different accelerators (GPU, MPS, etc).

2 Getting Started

To use Docling, you can simply install the *docling* package from PyPI. Documentation and examples are available in our GitHub repository at github.com/DS4SD/docling. All required model assets[1] are downloaded to a local huggingface datasets cache on first use, unless you choose to pre-install the model assets in advance.

Docling provides an easy code interface to convert PDF documents from file system, URLs or binary streams, and retrieve the output in either JSON or Markdown format. For convenience, separate methods are offered to convert single documents or batches of documents. A basic usage example is illustrated below. Further examples are available in the Doclign code repository.

```
from docling.document_converter import DocumentConverter

source = "https://arxiv.org/pdf/2206.01062"  # PDF path or URL
converter = DocumentConverter()
result = converter.convert_single(source)
print(result.render_as_markdown())  # output: "## DocLayNet: A Large
    Human-Annotated Dataset for Document-Layout Analysis [...]"
```

Optionally, you can configure custom pipeline features and runtime options, such as turning on or off features (e.g. OCR, table structure recognition), enforcing limits on the input document size, and defining the budget of CPU threads. Advanced usage examples and options are documented in the README file. Docling also provides a *Dockerfile* to demonstrate how to install and run it inside a container.

3 Processing pipeline

Docling implements a linear pipeline of operations, which execute sequentially on each given document (see Fig. 1). Each document is first parsed by a PDF backend, which retrieves the programmatic text tokens, consisting of string content and its coordinates on the page, and also renders a bitmap image of each page to support downstream operations. Then, the standard model pipeline applies a sequence of AI models independently on every page in the document to extract features and content, such as layout and table structures. Finally, the results from all pages are aggregated and passed through a post-processing stage, which augments metadata, detects the document language, infers reading-order and eventually assembles a typed document object which can be serialized to JSON or Markdown.

3.1 PDF backends

Two basic requirements to process PDF documents in our pipeline are a) to retrieve all text content and their geometric coordinates on each page and b) to render the visual representation of each page as it would appear in a PDF viewer. Both these requirements are encapsulated in Docling's PDF backend interface. While there are several open-source PDF parsing libraries available for python, we faced major obstacles with all of them for different reasons, among which were restrictive

[1] see huggingface.co/ds4sd/docling-models/

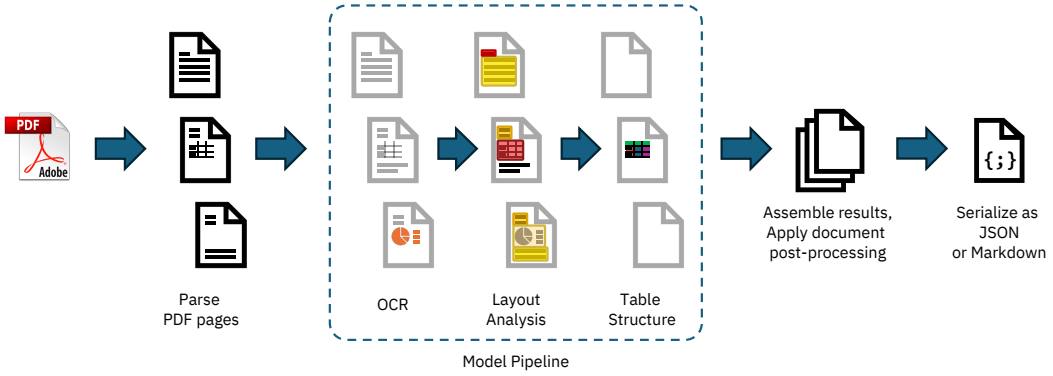


Figure 1: Sketch of Docling's default processing pipeline. The inner part of the model pipeline is easily customizable and extensible.

licensing (e.g. pymupdf [7]), poor speed or unrecoverable quality issues, such as merged text cells across far-apart text tokens or table columns (pypdfium, PyPDF) [15, 14].

We therefore decided to provide multiple backend choices, and additionally open-source a custom-built PDF parser, which is based on the low-level *qpdf* [4] library. It is made available in a separate package named *docling-parse* and powers the default PDF backend in Docling. As an alternative, we provide a PDF backend relying on *pypdfium*, which may be a safe backup choice in certain cases, e.g. if issues are seen with particular font encodings.

3.2 AI models

As part of Docling, we initially release two highly capable AI models to the open-source community, which have been developed and published recently by our team. The first model is a layout analysis model, an accurate object-detector for page elements [13]. The second model is TableFormer [12, 9], a state-of-the-art table structure recognition model. We provide the pre-trained weights (hosted on huggingface) and a separate package for the inference code as *docling-ibm-models*. Both models are also powering the open-access deepsearch-experience, our cloud-native service for knowledge exploration tasks.

Layout Analysis Model

Our layout analysis model is an object-detector which predicts the bounding-boxes and classes of various elements on the image of a given page. Its architecture is derived from RT-DETR [16] and re-trained on DocLayNet [13], our popular human-annotated dataset for document-layout analysis, among other proprietary datasets. For inference, our implementation relies on the *onnxruntime* [5].

The Docling pipeline feeds page images at 72 dpi resolution, which can be processed on a single CPU with sub-second latency. All predicted bounding-box proposals for document elements are post-processed to remove overlapping proposals based on confidence and size, and then intersected with the text tokens in the PDF to group them into meaningful and complete units such as paragraphs, section titles, list items, captions, figures or tables.

Table Structure Recognition

The TableFormer model [12], first published in 2022 and since refined with a custom structure token language [9], is a vision-transformer model for table structure recovery. It can predict the logical row and column structure of a given table based on an input image, and determine which table cells belong to column headers, row headers or the table body. Compared to earlier approaches, TableFormer handles many characteristics of tables, such as partial or no borderlines, empty cells, rows or columns, cell spans and hierarchy both on column-heading or row-heading level, tables with inconsistent indentation or alignment and other complexities. For inference, our implementation relies on *PyTorch* [2].

The Docling pipeline feeds all table objects detected in the layout analysis to the TableFormer model, by providing an image-crop of the table and the included text cells. TableFormer structure predictions are matched back to the PDF cells in post-processing to avoid expensive re-transcription text in the table image. Typical tables require between 2 and 6 seconds to be processed on a standard CPU, strongly depending on the amount of included table cells.

OCR

Docling provides optional support for OCR, for example to cover scanned PDFs or content in bitmaps images embedded on a page. In our initial release, we rely on *EasyOCR* [1], a popular third-party OCR library with support for many languages. Docling, by default, feeds a high-resolution page image (216 dpi) to the OCR engine, to allow capturing small print detail in decent quality. While EasyOCR delivers reasonable transcription quality, we observe that it runs fairly slow on CPU (upwards of 30 seconds per page).

We are actively seeking collaboration from the open-source community to extend Docling with additional OCR backends and speed improvements.

3.3 Assembly

In the final pipeline stage, Docling assembles all prediction results produced on each page into a well-defined datatype that encapsulates a converted document, as defined in the auxiliary package *docling-core*. The generated document object is passed through a post-processing model which leverages several algorithms to augment features, such as detection of the document language, correcting the reading order, matching figures with captions and labelling metadata such as title, authors and references. The final output can then be serialized to JSON or transformed into a Markdown representation at the users request.

3.4 Extensibility

Docling provides a straight-forward interface to extend its capabilities, namely the model pipeline. A model pipeline constitutes the central part in the processing, following initial document parsing and preceding output assembly, and can be fully customized by sub-classing from an abstract base-class (*BaseModelPipeline*) or cloning the default model pipeline. This effectively allows to fully customize the chain of models, add or replace models, and introduce additional pipeline configuration parameters. To use a custom model pipeline, the custom pipeline class to instantiate can be provided as an argument to the main document conversion methods. We invite everyone in the community to propose additional or alternative models and improvements.

Implementations of model classes must satisfy the python `Callable` interface. The `__call__` method must accept an iterator over page objects, and produce another iterator over the page objects which were augmented with the additional features predicted by the model, by extending the provided `PagePredictions` data model accordingly.

4 Performance

In this section, we establish some reference numbers for the processing speed of Docling and the resource budget it requires. All tests in this section are run with default options on our standard test set distributed with Docling, which consists of three papers from arXiv and two IBM Redbooks, with a total of 225 pages. Measurements were taken using both available PDF backends on two different hardware systems: one MacBook Pro M3 Max, and one bare-metal server running Ubuntu 20.04 LTS on an Intel Xeon E5-2690 CPU. For reproducibility, we fixed the thread budget (through setting *OMP_NUM_THREADS environment variable*) once to 4 (Docling default) and once to 16 (equal to full core count on the test hardware). All results are shown in Table 1.

If you need to run Docling in very low-resource environments, please consider configuring the pypdfium backend. While it is faster and more memory efficient than the default *docling-parse* backend, it will come at the expense of worse quality results, especially in table structure recovery.

Establishing GPU acceleration support for the AI models is currently work-in-progress and largely untested, but may work implicitly when CUDA is available and discovered by the onnxruntime and

torch runtimes backing the Docling pipeline. We will deliver updates on this topic at in a future version of this report.

Table 1: Runtime characteristics of Docling with the standard model pipeline and settings, on our test dataset of 225 pages, on two different systems. OCR is disabled. We show the time-to-solution (TTS), computed throughput in pages per second, and the peak memory used (resident set size) for both the Docling-native PDF backend and for the pypdfium backend, using 4 and 16 threads.

CPU	Thread budget	native backend			pypdfium backend		
		TTS	Pages/s	Mem	TTS	Pages/s	Mem
Apple M3 Max (16 cores)	4	177 s	1.27	6.20 GB	103 s	2.18	2.56 GB
	16	167 s	1.34		92 s	2.45	
Intel(R) Xeon E5-2690 (16 cores)	4	375 s	0.60	6.16 GB	239 s	0.94	2.42 GB
	16	244 s	0.92		143 s	1.57	

5 Applications

Thanks to the high-quality, richly structured document conversion achieved by Docling, its output qualifies for numerous downstream applications. For example, Docling can provide a base for detailed enterprise document search, passage retrieval or classification use-cases, or support knowledge extraction pipelines, allowing specific treatment of different structures in the document, such as tables, figures, section structure or references. For popular generative AI application patterns, such as retrieval-augmented generation (RAG), we provide *quackling*, an open-source package which capitalizes on Docling's feature-rich document output to enable document-native optimized vector embedding and chunking. It plugs in seamlessly with LLM frameworks such as LlamaIndex [8]. Since Docling is fast, stable and cheap to run, it also makes for an excellent choice to build document-derived datasets. With its powerful table structure recognition, it provides significant benefit to automated knowledge-base construction [11, 10]. Docling is also integrated within the open IBM data prep kit [6], which implements scalable data transforms to build large-scale multi-modal training datasets.

6 Future work and contributions

Docling is designed to allow easy extension of the model library and pipelines. In the future, we plan to extend Docling with several more models, such as a figure-classifier model, an equation-recognition model, a code-recognition model and more. This will help improve the quality of conversion for specific types of content, as well as augment extracted document metadata with additional information. Further investment into testing and optimizing GPU acceleration as well as improving the Docling-native PDF backend are on our roadmap, too.

We encourage everyone to propose or implement additional features and models, and will gladly take your inputs and contributions under review. The codebase of Docling is open for use and contribution, under the MIT license agreement and in alignment with our contributing guidelines included in the Docling repository. If you use Docling in your projects, please consider citing this technical report.

References

[1] J. AI. Easyocr: Ready-to-use ocr with 80+ supported languages. https://github.com/JaidedAI/EasyOCR, 2024. Version: 1.7.0.

[2] J. Ansel, E. Yang, H. He, N. Gimelshein, A. Jain, M. Voznesensky, B. Bao, P. Bell, D. Berard, E. Burovski, G. Chauhan, A. Chourdia, W. Constable, A. Desmaison, Z. DeVito, E. Ellison, W. Feng, J. Gong, M. Gschwind, B. Hirsh, S. Huang, K. Kalambarkar, L. Kirsch, M. Lazos, M. Lezcano, Y. Liang, J. Liang, Y. Lu, C. Luk, B. Maher, Y. Pan, C. Puhrsch, M. Reso, M. Saroufim, M. Y. Siraichi, H. Suk, M. Suo, P. Tillet, E. Wang, X. Wang, W. Wen, S. Zhang, X. Zhao, K. Zhou, R. Zou, A. Mathews, G. Chanan, P. Wu, and S. Chintala. Pytorch 2: Faster

machine learning through dynamic python bytecode transformation and graph compilation. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, 4 2024. doi: 10.1145/3620665.3640366. URL https://pytorch.org/assets/pytorch2-2.pdf.

[3] C. Auer, M. Dolfi, A. Carvalho, C. B. Ramis, and P. W. Staar. Delivering document conversion as a cloud service with high throughput and responsiveness. In *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*, pages 363–373. IEEE, 2022.

[4] J. Berkenbilt. Qpdf: A content-preserving pdf document transformer, 2024. URL https://github.com/qpdf/qpdf.

[5] O. R. developers. Onnx runtime. https://onnxruntime.ai/, 2024. Version: 1.18.1.

[6] IBM. Data Prep Kit: a community project to democratize and accelerate unstructured data preparation for LLM app developers, 2024. URL https://github.com/IBM/data-prep-kit.

[7] A. S. Inc. PyMuPDF, 2024. URL https://github.com/pymupdf/PyMuPDF.

[8] J. Liu. LlamaIndex, 11 2022. URL https://github.com/jerryjliu/llama_index.

[9] M. Lysak, A. Nassar, N. Livathinos, C. Auer, and P. Staar. Optimized Table Tokenization for Table Structure Recognition. In *Document Analysis and Recognition - ICDAR 2023: 17th International Conference, San José, CA, USA, August 21–26, 2023, Proceedings, Part II*, pages 37–50, Berlin, Heidelberg, Aug. 2023. Springer-Verlag. ISBN 978-3-031-41678-1. doi: 10.1007/978-3-031-41679-8_3. URL https://doi.org/10.1007/978-3-031-41679-8_3.

[10] L. Mishra, S. Dhibi, Y. Kim, C. Berrospi Ramis, S. Gupta, M. Dolfi, and P. Staar. Statements: Universal information extraction from tables with large language models for ESG KPIs. In D. Stammbach, J. Ni, T. Schimanski, K. Dutia, A. Singh, J. Bingler, C. Christiaen, N. Kushwaha, V. Muccione, S. A. Vaghefi, and M. Leippold, editors, *Proceedings of the 1st Workshop on Natural Language Processing Meets Climate Change (ClimateNLP 2024)*, pages 193–214, Bangkok, Thailand, Aug. 2024. Association for Computational Linguistics. URL https://aclanthology.org/2024.climatenlp-1.15.

[11] L. Morin, V. Weber, G. I. Meijer, F. Yu, and P. W. J. Staar. Patcid: an open-access dataset of chemical structures in patent documents. *Nature Communications*, 15(1):6532, August 2024. ISSN 2041-1723. doi: 10.1038/s41467-024-50779-y. URL https://doi.org/10.1038/s41467-024-50779-y.

[12] A. Nassar, N. Livathinos, M. Lysak, and P. Staar. Tableformer: Table structure understanding with transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4614–4623, 2022.

[13] B. Pfitzmann, C. Auer, M. Dolfi, A. S. Nassar, and P. Staar. Doclaynet: a large human-annotated dataset for document-layout segmentation. pages 3743–3751, 2022.

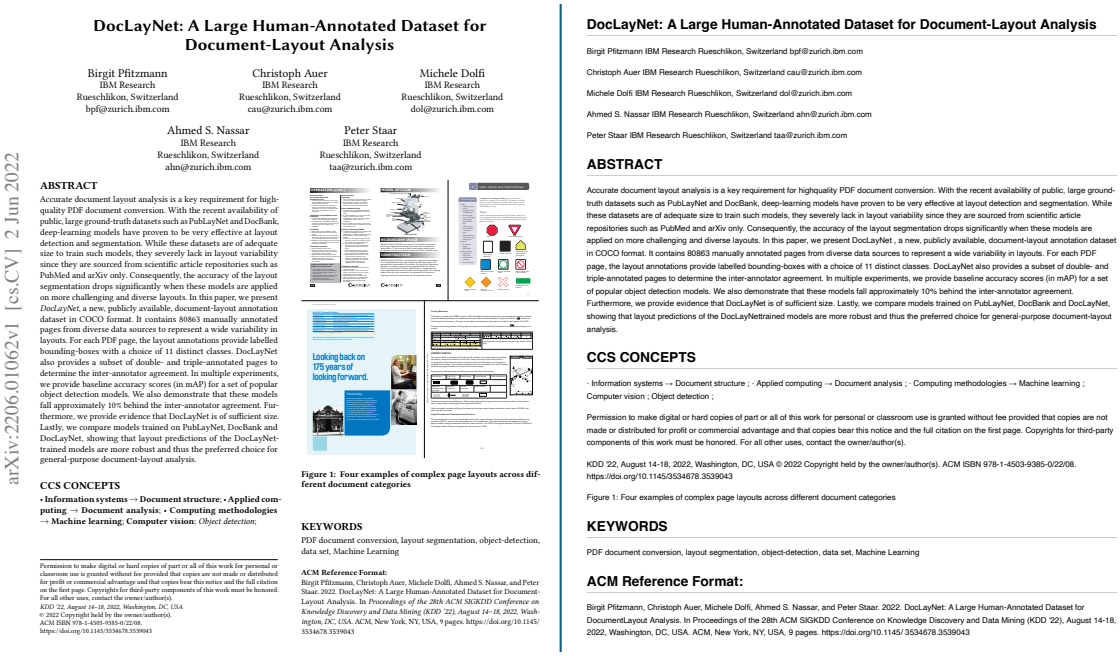
[14] pypdf Maintainers. pypdf: A Pure-Python PDF Library, 2024. URL https://github.com/py-pdf/pypdf.

[15] P. Team. PyPDFium2: Python bindings for PDFium, 2024. URL https://github.com/pypdfium2-team/pypdfium2.

[16] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen. Detrs beat yolos on real-time object detection, 2023.

Appendix

In this section, we illustrate a few examples of Docling's output in Markdown and JSON.



DocLayNet: A Large Human-Annotated Dataset for Document-Layout Analysis

Birgit Pfitzmann IBM Research Rueschlikon, Switzerland bpf@zurich.ibm.com

Christoph Auer IBM Research Rueschlikon, Switzerland cau@zurich.ibm.com

Michele Dolfi IBM Research Rueschlikon, Switzerland dol@zurich.ibm.com

Ahmed S. Nassar IBM Research Rueschlikon, Switzerland ahn@zurich.ibm.com

Peter Staar IBM Research Rueschlikon, Switzerland taa@zurich.ibm.com

ABSTRACT

Accurate document layout analysis is a key requirement for highquality PDF document conversion. With the recent availability of public, large ground-truth datasets such as PubLayNet and DocBank, deep-learning models have proven to be very effective at layout detection and segmentation. While these datasets are of adequate size to train such models, they severely lack in layout variability since they are sourced from scientific article repositories such as PubMed and arXiv only. Consequently, the accuracy of the layout segmentation drops significantly when these models are applied on more challenging and diverse layouts. In this paper, we present DocLayNet , a new, publicly available, document-layout annotation dataset in COCO format. It contains 80863 manually annotated pages from diverse data sources to represent a wide variability in layouts. For each PDF page, the layout annotations provide labelled bounding-boxes with a choice of 11 distinct classes. DocLayNet also provides a subset of double- and triple-annotated pages to determine the inter-annotator agreement. In multiple experiments, we provide baseline accuracy scores (in mAP) for a set of popular object detection models. We also demonstrate that these models fall approximately 10% behind the inter-annotator agreement. Furthermore, we provide evidence that DocLayNet is of sufficient size. Lastly, we compare models trained on PubLayNet, DocBank and DocLayNet, showing that layout predictions of the DocLayNettrained models are more robust and thus the preferred choice for general-purpose document-layout analysis.

CCS CONCEPTS

· Information systems → Document structure ; · Applied computing → Document analysis ; · Computing methodologies → Machine learning ; Computer vision ; Object detection ;

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

KDD '22, August 14-18, 2022, Washington, DC, USA © 2022 Copyright held by the owner/author(s). ACM ISBN 978-1-4503-9385-0/22/08. https://doi.org/10.1145/3534678.3539043

Figure 1: Four examples of complex page layouts across different document categories

KEYWORDS

PDF document conversion, layout segmentation, object-detection, data set, Machine Learning

ACM Reference Format:

Birgit Pfitzmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar. 2022. DocLayNet: A Large Human-Annotated Dataset for DocumentLayout Analysis. In Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '22), August 14-18, 2022, Washington, DC, USA. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/ 3534678.3539043

Figure 2: Title page of the DocLayNet paper (arxiv.org/pdf/2206.01062) - left PDF, right rendered Markdown. If recognized, metadata such as authors are appearing first under the title. Text content inside figures is currently dropped, the caption is retained and linked to the figure in the JSON representation (not shown).

Table 2: Prediction performance (mAP@0.5-0.95) of object detection networks on DocLayNet test set. The MRCNN (Mask R-CNN) and FRCNN (Faster R-CNN) models with ResNet-50 or ResNet-101 backbone were trained based on the network architectures from the *detectron2* model zoo (Mask R-CNN R50, R101-FPN 3x, Faster R-CNN R101-FPN 3x), with default configurations. The YOLO implementation utilized was YOLOv5x6 [13]. All models were initialised using pre-trained weights from the COCO 2017 dataset.

	human	MRCNN		FRCNN	YOLO
		R50	R101	R101	v5x6
Caption	84-89	68.4	71.5	70.1	77.7
Footnote	83-91	70.9	71.8	73.7	77.2
Formula	83-85	60.1	63.4	63.5	66.2
List-item	87-88	81.2	80.8	81.0	86.2
Page-footer	93-94	61.6	59.3	58.9	61.1
Page-header	85-89	71.9	70.0	72.0	67.9
Picture	69-71	71.7	72.7	72.0	77.1
Section-header	83-84	67.6	69.3	68.4	74.6
Table	77-81	82.2	82.9	82.2	86.3
Text	84-86	84.6	85.8	85.4	88.1
Title	60-72	76.7	80.4	79.9	82.7
All	82-83	72.4	73.5	73.4	76.8

to avoid this at any cost in order to have clear, unbiased baseline numbers for human document-layout annotation. Third, we introduced the feature of *snapping* boxes around text segments to obtain a pixel-accurate annotation and again reduce time and effort. The CCS annotation tool automatically shrinks every user-drawn box to the minimum bounding-box around the enclosed text-cells for all purely text-based segments, which excludes only *Table* and *Picture*. For the latter, we instructed annotation staff to minimise inclusion of surrounding whitespace while including all graphical lines. A downside of snapping boxes to enclosed text cells is that some wrongly parsed PDF pages cannot be annotated correctly and need to be skipped. Fourth, we established a way to flag pages as *rejected* for cases where no valid annotation according to the label guidelines could be achieved. Example cases for this would be PDF pages that render incorrectly or contain layouts that are impossible to capture with non-overlapping rectangles. Such rejected pages are not contained in the final dataset. With all these measures in place, experienced annotation staff managed to annotate a single page in a typical timeframe of 20s to 60s, depending on its complexity.

5 EXPERIMENTS

The primary goal of DocLayNet is to obtain high-quality ML models capable of accurate document-layout analysis on a wide variety of challenging layouts. As discussed in Section 2, object detection models are currently the easiest to use, due to the standardisation of ground-truth data in COCO format [16] and the availability of general frameworks such as *detectron2* [17]. Furthermore, baseline numbers in PubLayNet and DocBank were obtained using standard object detection models such as Mask R-CNN and Faster R-CNN. As such, we will relate to these object detection methods in this

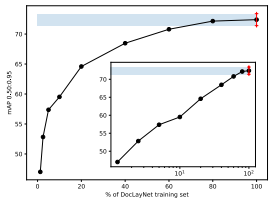


Figure 5: Prediction performance (mAP@0.5-0.95) of a Mask R-CNN network with ResNet50 backbone trained on increasing fractions of the DocLayNet dataset. The learning curve flattens around the 80% mark, indicating that increasing the size of the DocLayNet dataset with similar data will not yield significantly better predictions.

paper and leave the detailed evaluation of more recent methods mentioned in Section 2 for future work.

In this section, we will present several aspects related to the performance of object detection models on DocLayNet. Similarly as in PubLayNet, we will evaluate the quality of their predictions using mean average precision (mAP) with 10 overlaps that range from 0.5 to 0.95 in steps of 0.05 (mAP@0.5-0.95). These scores are computed by leveraging the evaluation code provided by the COCO API [16].

Baselines for Object Detection

In Table 2, we present baseline experiments (given in mAP) on Mask R-CNN [12], Faster R-CNN [11], and YOLOv5 [13]. Both training and evaluation were performed on RGB images with dimensions of 1025×1025 pixels. For training, we only used one annotation in case of redundantly annotated pages. As one can observe, the variation in mAP between the models is rather low, but overall between 6 and 10% lower than the mAP computed from the pairwise human annotations on triple-annotated pages. This gives a good indication that the DocLayNet dataset poses a worthwhile challenge for the research community to close the gap between human recognition and ML approaches. It is interesting to see that Mask R-CNN and Faster R-CNN produce very comparable mAP scores, indicating that pixel-based image segmentation derived from bounding-boxes does not help to obtain better predictions. On the other hand, the more recent Yolov5x model does very well and even out-performs humans on selected labels such as *Text*, *Table* and *Picture*. This is not entirely surprising, as *Text*, *Table* and *Picture* are abundant and the most visually distinctive in a document.

Table 2: Prediction performance (mAP@0.5-0.95) of object detection networks on DocLayNet test set. The MRCNN (Mask R-CNN) and FRCNN (Faster R-CNN) models with ResNet-50 or ResNet-101 backbone were trained based on the network architectures from the detectron2 model zoo (Mask R-CNN R50, R101-FPN 3x, Faster R-CNN R101-FPN 3x), with default configurations. The YOLO implementation utilized was YOLOv5x6 [13]. All models were initialised using pre-trained weights from the COCO 2017 dataset.

	human	MRCNN	MRCNN	FRCNN	YOLO
	human	R50	R101	R101	v5x6
Caption	84-89	68.4	71.5	70.1	77.7
Footnote	83-91	70.9	71.8	73.7	77.2
Formula	83-85	60.1	63.4	63.5	66.2
List-item	87-88	81.2	80.8	81.0	86.2
Page-footer	93-94	61.6	59.3	58.9	61.1
Page-header	85-89	71.9	70.0	72.0	67.9
Picture	69-71	71.7	72.7	72.0	77.1
Section-header	83-84	67.6	69.3	68.4	74.6
Table	77-81	82.2	82.9	82.2	86.3
Text	84-86	84.6	85.8	85.4	88.1
Title	60-72	76.7	80.4	79.9	82.7
All	82-83	72.4	73.5	73.4	76.8

to avoid this at any cost in order to have clear, unbiased baseline numbers for human document-layout annotation. Third, we introduced the feature of snapping boxes around text segments to obtain a pixel-accurate annotation and again reduce time and effort. The CCS annotation tool automatically shrinks every user-drawn box to the minimum bounding-box around the enclosed text-cells for all purely text-based segments, which excludes only Table and Picture . For the latter, we instructed annotation staff to minimise inclusion of surrounding whitespace while including all graphical lines. A downside of snapping boxes to enclosed text cells is that some wrongly parsed PDF pages cannot be annotated correctly and need to be skipped. Fourth, we established a way to flag pages as rejected for cases where no valid annotation according to the label guidelines could be achieved. Example cases for this would be PDF pages that render incorrectly or contain layouts that are impossible to capture with non-overlapping rectangles. Such rejected pages are not contained in the final dataset. With all these measures in place, experienced annotation staff managed to annotate a single page in a typical timeframe of 20s to 60s, depending on its complexity.

5 EXPERIMENTS

The primary goal of DocLayNet is to obtain high-quality ML models capable of accurate document-layout analysis on a wide variety of challenging layouts. As discussed in Section 2, object detection models are currently the easiest to use, due to the standardisation of ground-truth data in COCO format [16] and the availability of general frameworks such as detectron2 [17]. Furthermore, baseline numbers in PubLayNet and DocBank were obtained using standard object detection models such as Mask R-CNN and Faster R-CNN. As such, we will relate to these object detection methods in this

Figure 5: Prediction performance (mAP@0.5-0.95) of a Mask R-CNN network with ResNet50 backbone trained on increasing fractions of the DocLayNet dataset. The learning curve flattens around the 80% mark, indicating that increasing the size of the DocLayNet dataset with similar data will not yield significantly better predictions.

paper and leave the detailed evaluation of more recent methods mentioned in Section 2 for future work.

In this section, we will present several aspects related to the performance of object detection models on DocLayNet. Similarly as in PubLayNet, we will evaluate the quality of their predictions using mean average precision (mAP) with 10 overlaps that range from 0.5 to 0.95 in steps of 0.05 (mAP@0.5-0.95). These scores are computed by leveraging the evaluation code provided by the COCO API [16].

Baselines for Object Detection

In Table 2, we present baseline experiments (given in mAP) on Mask R-CNN [12], Faster R-CNN [11], and YOLOv5 [13]. Both training and evaluation were performed on RGB images with dimensions of 1025 × 1025 pixels. For training, we only used one annotation in case of redundantly annotated pages. As one can observe, the variation in mAP between the models is rather low, but overall between 6 and 10% lower than the mAP computed from the pairwise human annotations on triple-annotated pages. This gives a good indication that the DocLayNet dataset poses a worthwhile challenge for the research community to close the gap between human recognition and ML approaches. It is interesting to see that Mask R-CNN and Faster R-CNN produce very comparable mAP scores, indicating that pixel-based image segmentation derived from bounding-boxes does not help to obtain better predictions. On the other hand, the more recent Yolov5x model does very well and even out-performs humans on selected labels such as Text , Table and Picture . This is not entirely surprising, as Text , Table and Picture are abundant and the most visually distinctive in a document.

Figure 3: Page 6 of the DocLayNet paper. If recognized, metadata such as authors are appearing first under the title. Elements recognized as page headers or footers are suppressed in Markdown to deliver uninterrupted content in reading order. Tables are inserted in reading order. The paragraph in "5. Experiments" wrapping over the column end is broken up in two and interrupted by the table.

A

class label	Count	% of Total			triple inter-annotator mAP @ 0.5-0.95 (%)						
		Train	Test	Val	All	Fin	Man	Sci	Law	Pat	Ten
Caption	22524	2.04	1.77	2.32	84-89	40-61	86-92	94-99	95-99	69-78	n/a
Footnote	6318	0.60	0.31	0.58	83-91	n/a	100	62-88	85-94	n/a	82-97
Formula	25027	2.25	1.90	2.96	83-85	n/a	n/a	84-87	86-96	n/a	n/a
List-item	185660	17.19	13.34	15.82	87-88	74-83	90-92	97-97	81-85	75-88	93-95
Page-footer	70878	6.51	5.58	6.00	93-94	88-90	95-96	100	92-97	100	96-98
Page-header	58022	5.10	6.70	5.06	85-89	66-76	90-94	98-100	91-92	97-99	81-86
Picture	45976	4.21	2.78	5.31	69-71	56-59	82-86	69-82	80-95	66-71	59-76
Section-header	142884	12.60	15.77	12.85	83-84	76-81	90-92	94-95	87-94	69-73	78-86
Table	34733	3.20	2.27	3.60	77-81	75-80	83-86	98-99	58-80	79-84	70-85
Text	510377	45.82	49.28	45.00	84-86	81-86	88-93	89-93	87-92	71-79	87-95
Title	5071	0.47	0.30	0.50	60-72	24-63	50-63	94-100	82-96	68-79	24-56
Total	1107470	941123	99816	66531	82-83	71-74	79-81	89-94	86-91	71-76	68-85

B

		% of Total	% of Total	% of Total	triple inter-annotator mAP @ 0.5-0.95 (%)						
class label	Count	Train	Test	Val	All	Fin	Man	Sci	Law	Pat	Ten
Caption	22524	2.04	1.77	2.32	84-89	40-61	86-92	94-99	95-99	69-78	n/a
Footnote	6318	0.60	0.31	0.58	83-91	n/a	100	62-88	85-94	n/a	82-97
Formula	25027	2.25	1.90	2.96	83-85	n/a	n/a	84-87	86-96	n/a	n/a
List-item	185660	17.19	13.34	15.82	87-88	74-83	90-92	97-97	81-85	75-88	93-95
Page-footer	70878	6.51	5.58	6.00	93-94	88-90	95-96	100	92-97	100	96-98
Page-header	58022	5.10	6.70	5.06	85-89	66-76	90-94	98-100	91-92	97-99	81-86
Picture	45976	4.21	2.78	5.31	69-71	56-59	82-86	69-82	80-95	66-71	59-76
Section-header	142884	12.60	15.77	12.85	83-84	76-81	90-92	94-95	87-94	69-73	78-86
Table	34733	3.20	2.27	3.60	77-81	75-80	83-86	98-99	58-80	79-84	70-85
Text	510377	45.82	49.28	45.00	84-86	81-86	88-93	89-93	87-92	71-79	87-95
Title	5071	0.47	0.30	0.50	60-72	24-63	50-63	94-100	82-96	68-79	24-56
Total	1107470	941123	99816	66531	82-83	71-74	79-81	89-94	86-91	71-76	68-85

C


```
"tables": [
    {
        "#-cols": 12,
        "#-rows": 14,
        "data": [
            {
                "bbox": [
                    329.04998779296875,
                    643.40185546875,
                    483.39764404296875,
                    651.7764892578125
                ],
                "spans": [...
                ],
                "text": "triple inter-annotator mAP @ 0.5-0.95 (%)",
                "col": 5,
                "col-span": [5,12],
                "row": 0,
                "row-span": [0,1]
            },
            # ...
        ]}]
```


Figure 4: Table 1 from the DocLayNet paper in the original PDF (A), as rendered Markdown (B) and in JSON representation (C). Spanning table cells, such as the multi-column header "triple inter-annotator mAP@0.5-0.95 (%)", is repeated for each column in the Markdown representation (B), which guarantees that every data point can be traced back to row and column headings only by its grid coordinates in the table. In the JSON representation, the span information is reflected in the fields of each table cell (C).